GitHub Actions OIDC Setup Guide For Beginners

Storing static cloud API keys in GitHub secrets creates severe security risks if keys leak or fail to rotate. OIDC solves this by letting GitHub securely trade short-lived tokens for cloud access on the fly.

What you'll learn:

1. 1. Grant Workflow Token Permissions
2. 2. Configure the IAM Trust Policy
3. 3. Assume the Role in GitHub Actions
4. 4. Verify Active Cloud Identity

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Grant Workflow Token Permissions

GitHub Actions disables OpenID Connect token minting by default for security. You must explicitly give the workflow id-token write permission at the job or workflow level. Without this permission block, GitHub will not generate the cryptographic JWT token, causing the cloud login step to fail immediately.

```
permissions:
  id-token: write
  contents: read
```

GitHub Actions OIDC: Connect to Cloud Without Storing Secrets

Sunday, October 18, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Configure the IAM Trust Policy

AWS IAM needs to know which specific repository is allowed to assume your deploy role. You configure this using a trust policy condition based on the token subject (sub claim). Specifying the exact repository name and branch prevents unauthorized users and external forks from accessing your cloud.

```
${
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercont
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:sub": "repo:myorg/myrepo:ref:refs/heads/main"
      }
    }
  }]
}
```

GitHub Actions OIDC: Connect to Cloud Without Storing Secrets

Sunday, October 18, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Assume the Role in GitHub Actions

Use the official AWS credentials action inside your workflow steps to complete the handshake. Pass your target AWS Role ARN and region without providing any static secret keys. The action automatically reads GitHub's OIDC token and returns temporary session keys valid for your job.

```
$ name: Configure AWS Credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123456789012:role/GitHubDeployRole
    aws-region: us-east-1
    role-duration-seconds: 900
```

GitHub Actions OIDC: Connect to Cloud Without Storing Secrets

Sunday, October 18, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Verify Active Cloud Identity

Always run a quick verification command directly after assuming the role. The AWS STS caller identity command outputs your temporary role session and account ID. This proves your runner is authenticated before it attempts to deploy resources or sync files.

```
- name: Verify Authentication
  run: |
    aws sts get-caller-identity
    aws s3 ls
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always set role-duration-seconds to the minimum time your job actually needs.
+ Pin your trust policy condition to specific branches like refs/heads/main.
+ Delete old static IAM user access keys once OIDC is working.
+ Keep OIDC permissions scoped only to the specific deployment job, not the whole file.
+ Set up separate IAM roles for staging and production deployment workflows.

Watch Out For

! Forgetting to add 'id-token: write' in workflow permissions, leading to 403 errors.
! Using a wildcard '*' in the trust policy sub claim, allowing any GitHub repo to assume your role.
! Typing the repository name incorrectly in the trust policy (it is strictly case-sensitive).

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh